Agent Loop Architecture

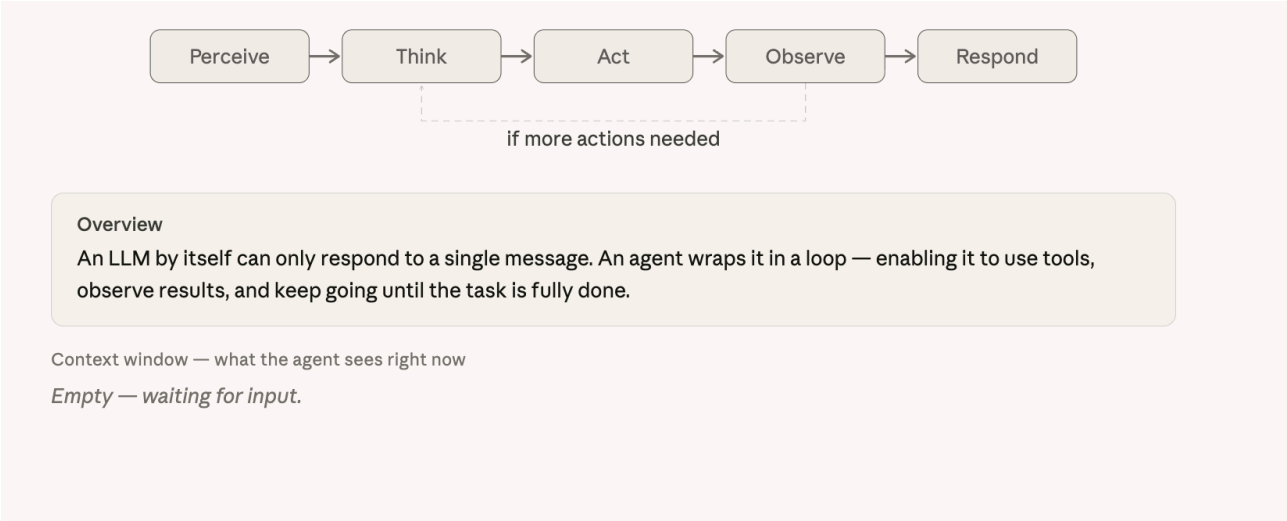
The core idea — a plain LLM is stateless: one message in, one message out, done. An agent loop breaks that constraint. It wraps the LLM in a repeating cycle so the model can act, check what happened, and keep going.

What each phase does:

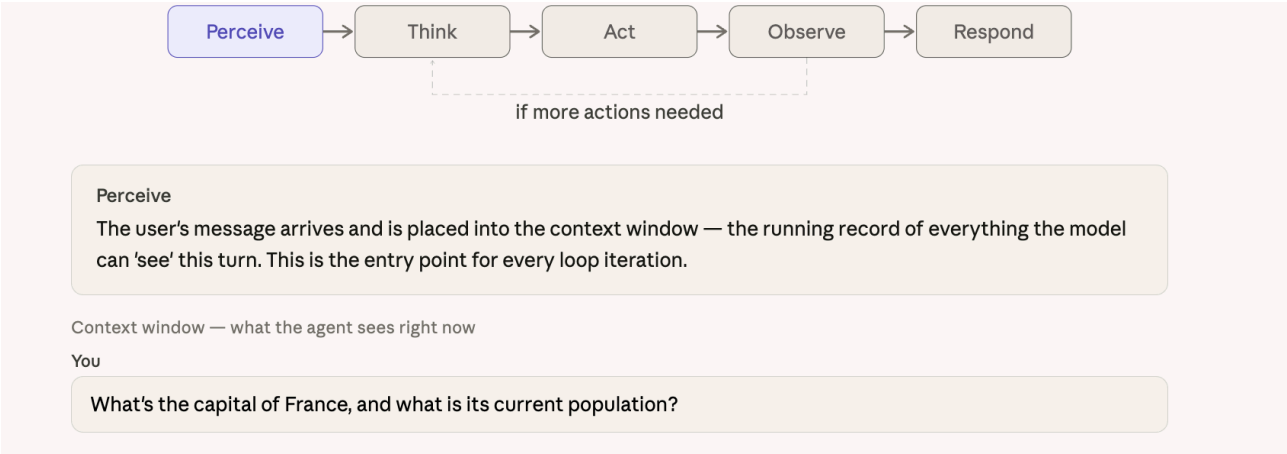
- **Perceive** — new input arrives (user message, scheduled trigger, another agent's output). It's appended to the context window.
- **Think** — the LLM reads everything in the context window and decides what to do next: call a tool, or produce a final response.
- **Act** — if a tool is needed, the model emits a structured tool call (web search, code execution, file read, API call, etc.). Control leaves the model.
- **Observe** — the tool result comes back and is appended to the context. Now the loop repeats from Think.
- **Respond** — when the model decides it has enough information, it outputs a final answer and the loop exits.

Why this matters — the loop is what makes agents *capable*. A single LLM call can write a paragraph. An agent loop can browse the web, write and run code, read files, check its own output, and retry on failure — all autonomously.

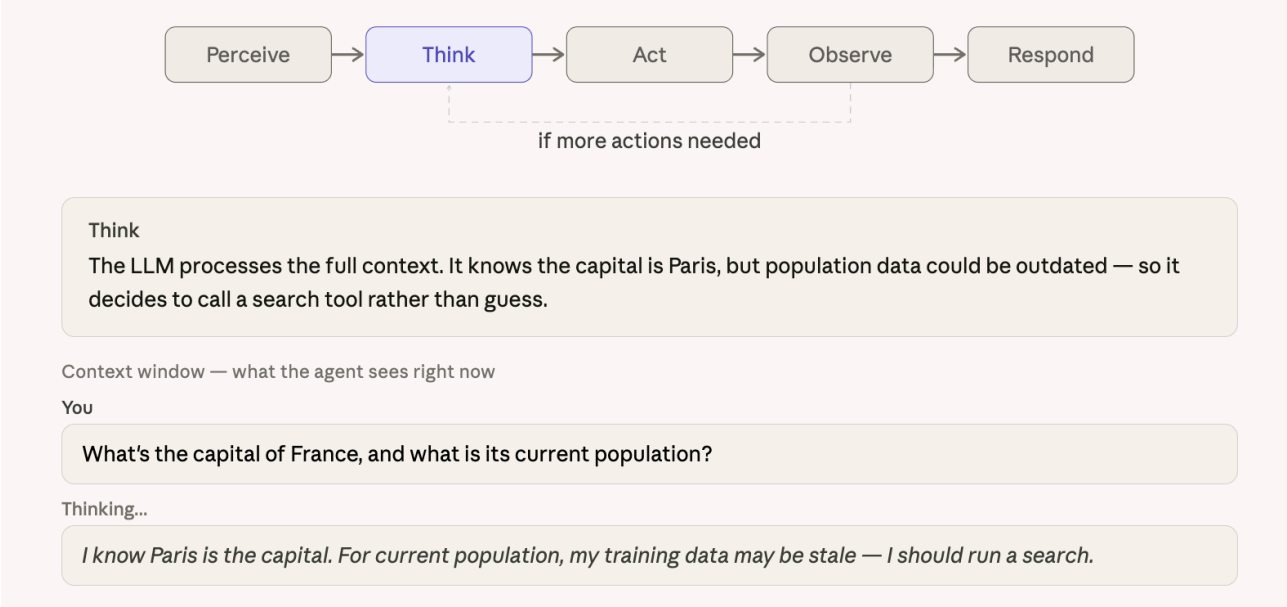
The Zero state



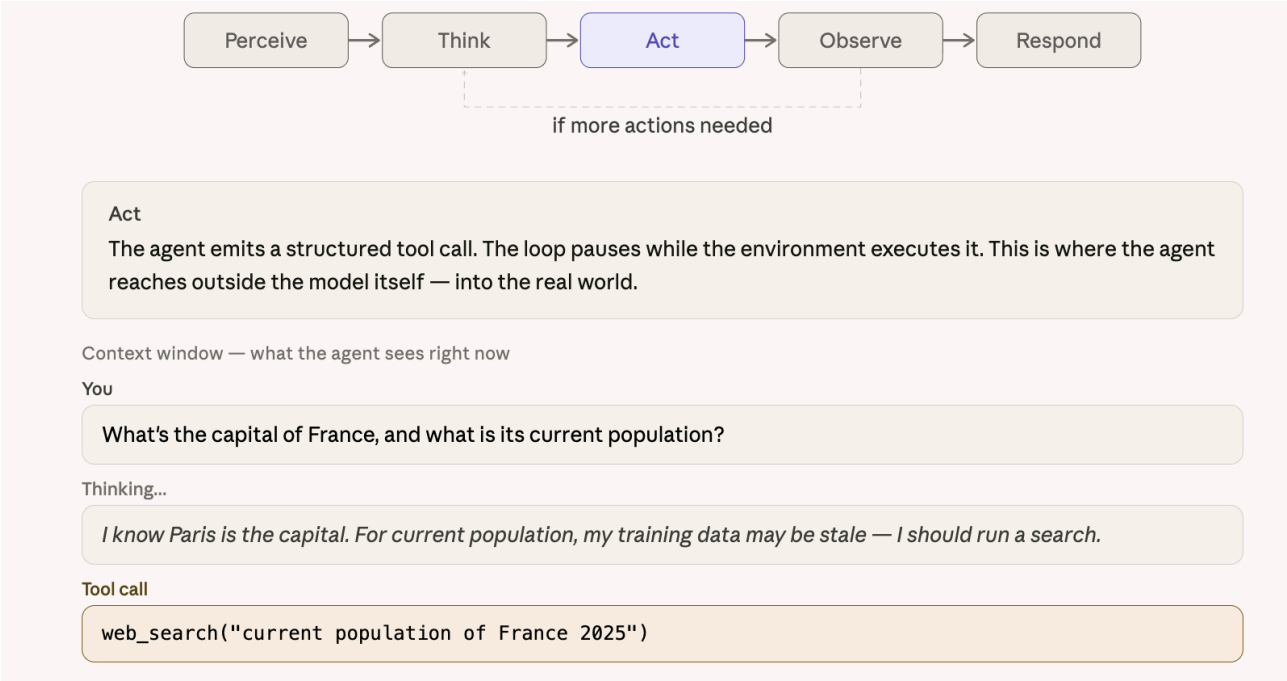
State 1



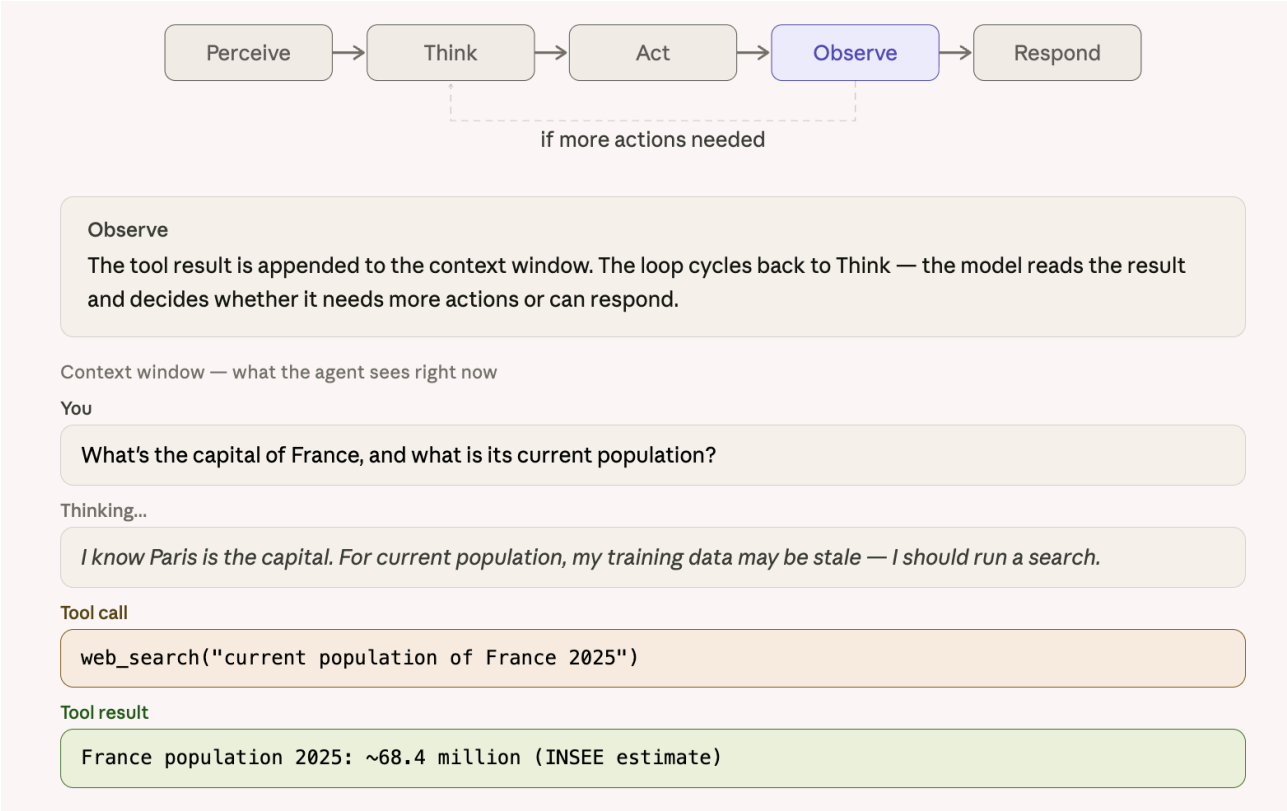
State 2



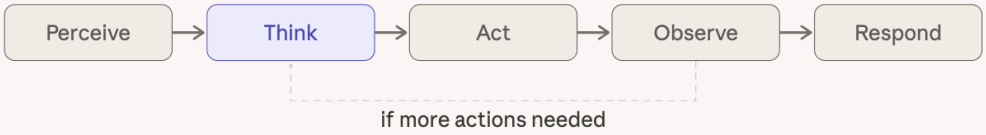
State 3



State 4



State 5



Think (again)
Back in the Think phase — this is the loop repeating. The LLM now has all it needs. It decides no more tools are required and moves toward generating a final answer.

Context window — what the agent sees right now

You

What's the capital of France, and what is its current population?

Thinking...

I know Paris is the capital. For current population, my training data may be stale — I should run a search.

Tool call

```
web_search("current population of France 2025")
```

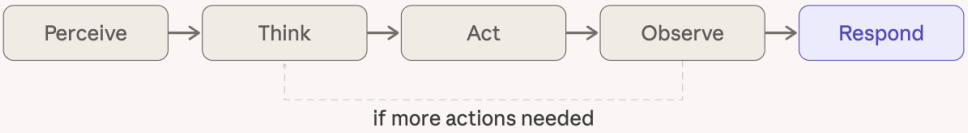
Tool result

```
France population 2025: ~68.4 million (INSEE estimate)
```

Thinking...

I have both answers now. No more tools needed — ready to respond.

State 6



Respond
The agent produces its final response and exits the loop. On harder tasks this loop could run for dozens of turns, calling different tools, writing code, reading files, or reflecting on earlier results.

Context window — what the agent sees right now

You

What's the capital of France, and what is its current population?

Thinking...

I know Paris is the capital. For current population, my training data may be stale — I should run a search.

Tool call

```
web_search("current population of France 2025")
```

Tool result

```
France population 2025: ~68.4 million (INSEE estimate)
```

Thinking...

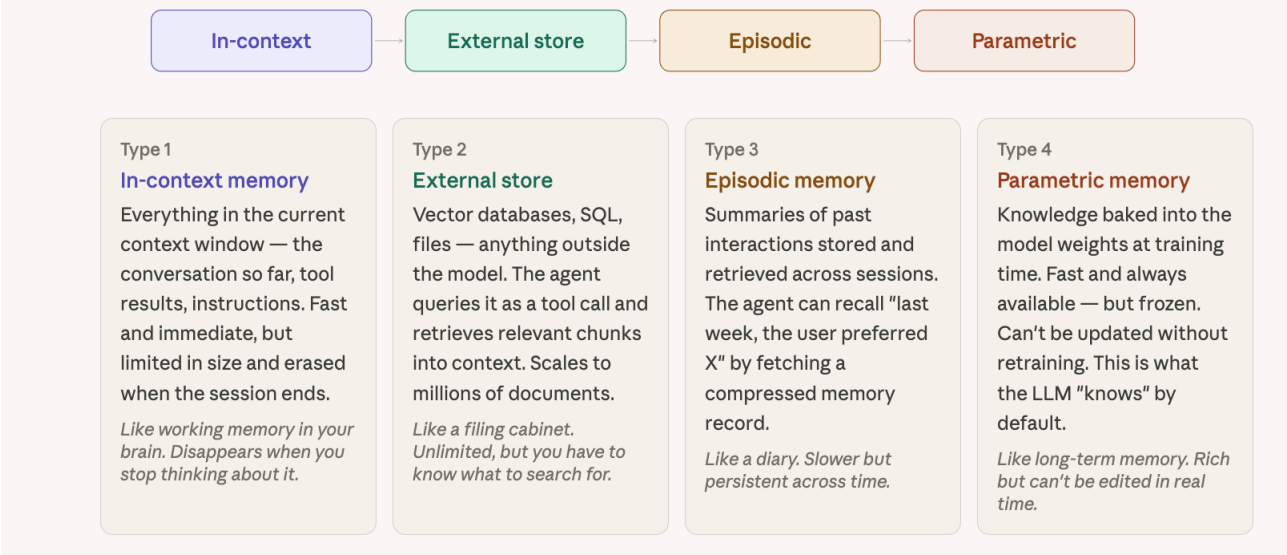
I have both answers now. No more tools needed — ready to respond.

Agent

The capital of France is Paris. As of 2025, its population is approximately 68.4 million people.

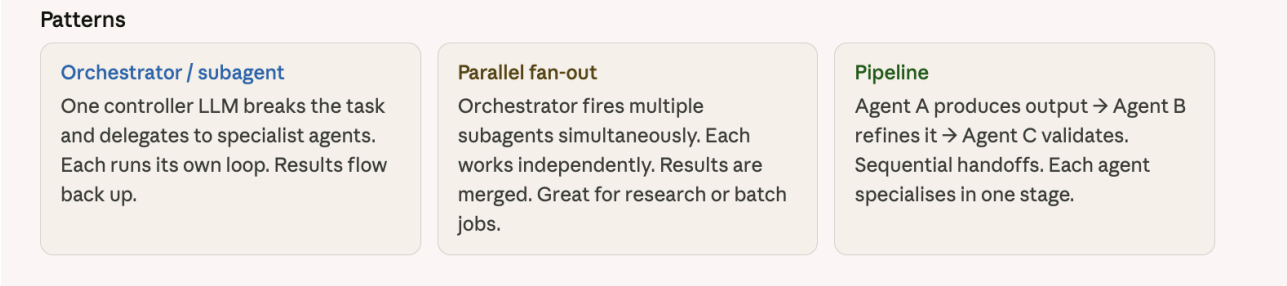
There are three layers that turn a basic loop into a production agent system.

1. Memory

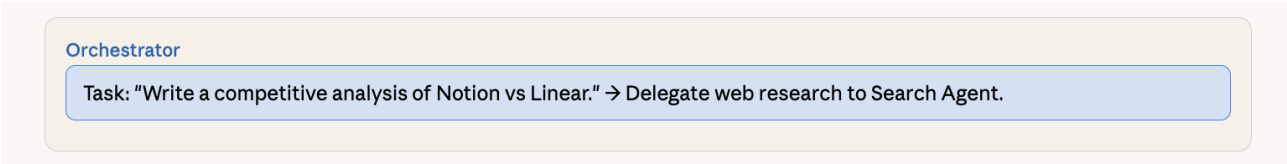


2. Multi Agent

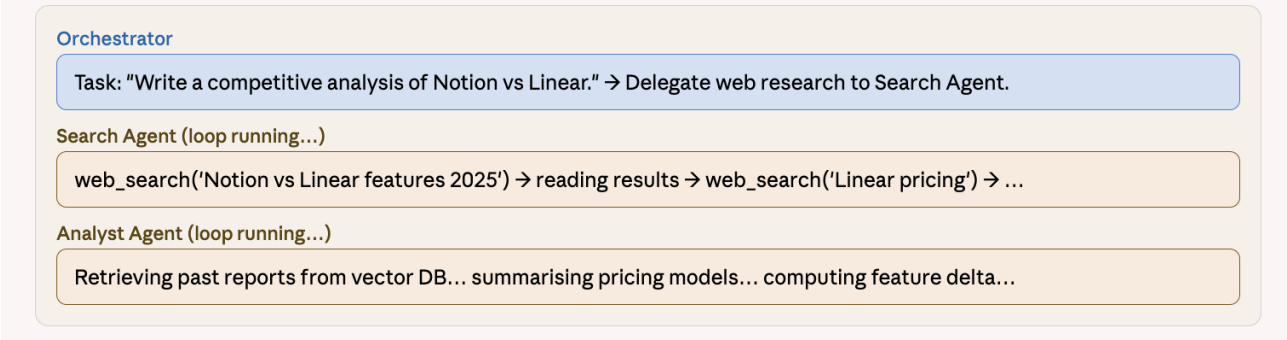
A single orchestrator agent receives the user's task. It has one job: break the work into subtasks and delegate — it won't do the research itself.



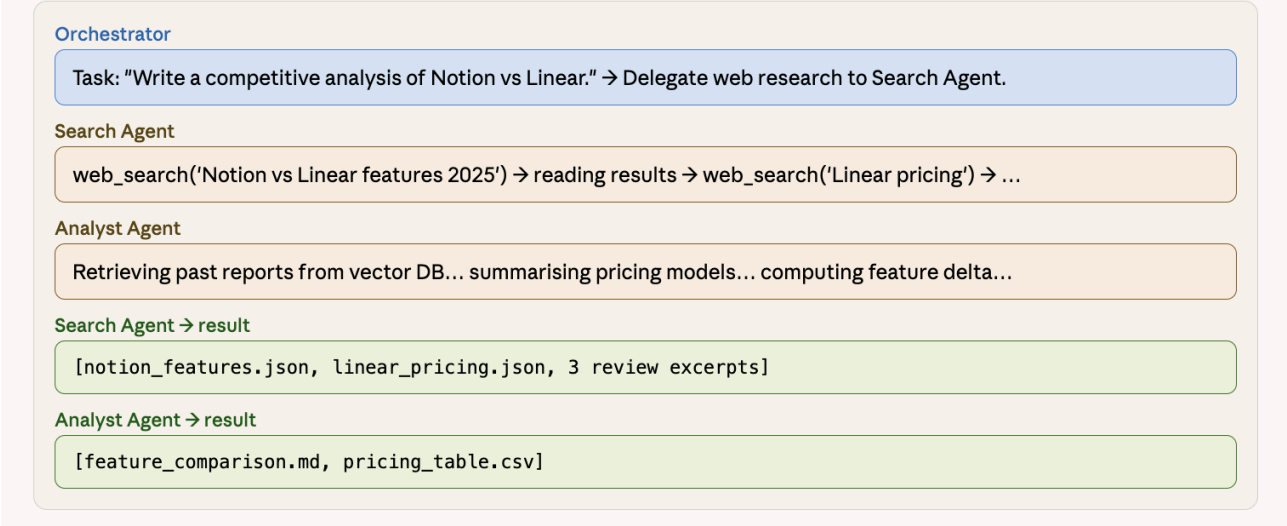
1. the orchestrator decomposes the task and fires a Search Agent as a subagent. That subagent runs its own full agent loop — perceive, think, act, observe — independently.



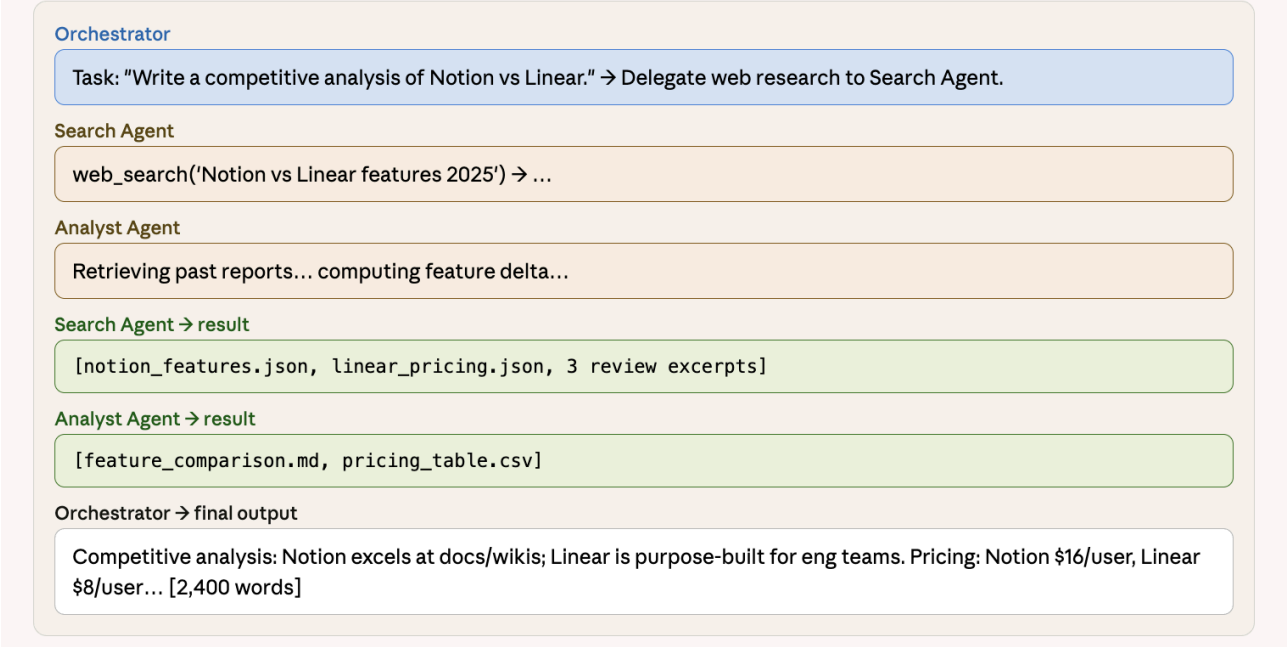
2. In parallel, a second subagent — the Analyst — is spun up. Both agents run concurrently. Neither knows about the other; they just produce results.



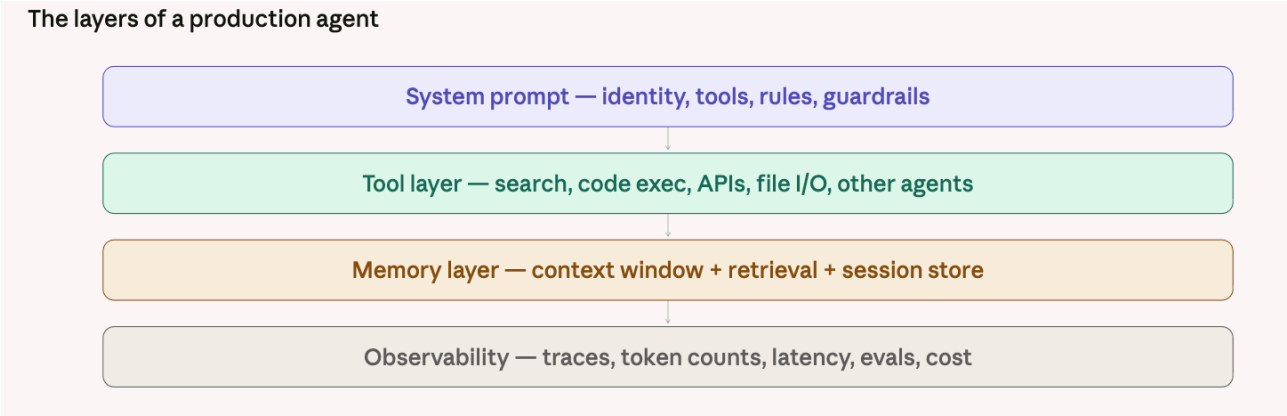
3. Both subagents return their results to the orchestrator's context window. The orchestrator now has rich, fresh data — far more than would fit in a single loop.



4. The orchestrator synthesises everything and produces the final output. No single agent needed to hold all that context — the system handled it across agents.



3. Production

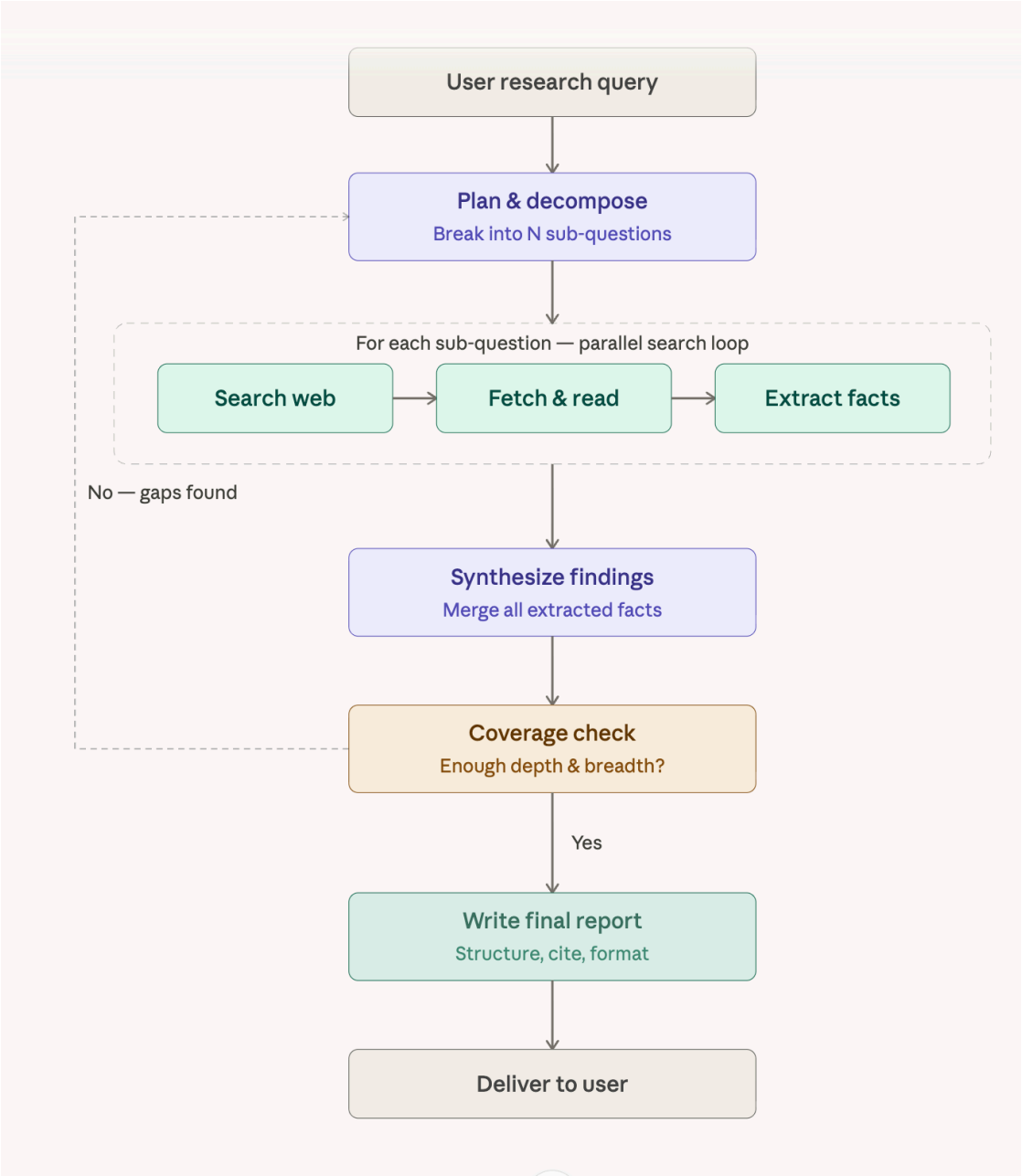


Memory is what makes agents stateful. Without it, every session starts from zero. Real agents layer all four types — using the context window for the current task, vector retrieval for documents and history, and the model's parametric knowledge as a baseline.

Multi-agent systems are what make agents scalable. A single agent's context window is a hard ceiling. If your task requires more information than fits, or has independent subtasks that could run in parallel, you need to distribute the work. The orchestrator/subagent pattern is the most common architecture — one "manager" LLM that delegates to specialist workers, each running its own loop.

Production hardening is the gap between a demo and a real system. The loop itself is simple. What's hard is: what happens when the model calls a tool with wrong arguments? What if the loop never terminates? What if a subagent takes a destructive action? Production teams invest heavily in guardrails, max-turn budgets, human approval checkpoints for irreversible actions, and tracing every single loop turn for debugging.

The underlying mental model: **every capability you add to an agent (memory, tools, other agents) increases what it can do but also increases what can go wrong**. The best production agents are conservative by default — they confirm before acting, summarise before overflowing context, and surface uncertainty rather than guessing.



A few things worth noting about how this differs from a basic agent loop:

The dashed container is the key addition. A basic loop calls one tool and waits. A deep research agent fans out — it generates N sub-questions upfront (e.g. "what are the main competitors", "what do users say", "what's the pricing landscape") and runs a full search-fetch-extract cycle for each, often in parallel. This is what lets it cover a topic at breadth before synthesizing.

The loop-back is deliberate, not a failure. After synthesizing, the coverage check isn't just "did I get results" — it's reflective reasoning: *do I have enough?* Are there contradictions? Missing angles? If yes to gaps, it re-enters the planning phase to generate new sub-questions targeting those gaps. Real agents like Perplexity Deep Research or OpenAI's deep research mode typically run 3–10 of these outer loops before writing.

Two loops are nested here. The inner loop (search → fetch → extract) runs per sub-question. The outer loop (plan → search → synthesize → check) runs until coverage is satisfied. This nesting is what makes deep research expensive — 5 outer iterations × 8 sub-questions × 3 tool calls each = 120 tool calls before a single word of the report is written.

The write step comes last and only once. A common mistake in naive implementations is writing incrementally. Production agents defer writing until all research is done — this lets the synthesis step inform the report structure rather than retrofitting conclusions onto a pre-written skeleton.

What makes production agents hard

Context overflow | Hallucinated tool calls | Infinite loops | Irreversible actions | Latency per loop turn | Cost at scale

How teams solve them

Context compression / summarise-and-forget | Max-turn limits + circuit breakers | Human-in-the-loop checkpoints | Dry-run / confirmation before writes | Structured outputs to force valid tool calls | Async subagent execution | Eval harnesses (automated regression testing) | Tracing every loop turn (LangSmith, Langfuse, etc.)